

IMPLEMENTATION :

4.1 Target Environment

The gateway was deployed on a production IBM LSF 9.1 cluster running RHEL 6.4 (kernel 2.6.32) comprising 32 nodes (22 active). Pre-deployment security assessment identified 15+ unpatched CVEs including CVE-2014-0160 (Heartbleed) and CVE-2016-0777 (SSH key leak)

4.2 Technology Stack

The gateway backend was implemented using FastAPI 0.115 on Python 3.12. Authentication was handled via Keycloak 26 using RS256 tokens, and PostgreSQL 16 was used to store databases and enforce policies. The frontend was built with React 18, and connectivity to the cluster was achieved through Paramiko SSH/SFTP. All services were containerized using Docker Compose with Nginx TLS 1.3 as a reverse proxy, and audit logs were hashed using HMAC-SHA256.

4.3 Three-Layer Code Defense

The gateway protects submitted Python scripts using three layers of defense, each targeting a different type of attack.

Layer 1 Static Analysis (AST Scanner):

Before a script runs, it is checked for dangerous code. The scanner looks at all import statements, restricted attributes, risky built-in functions like `eval` or `exec`, and special strings that could bypass security. Only safe modules like NumPy, pandas, PyTorch, and MPI libraries are allowed.

Layer 2 Runtime Sandbox:

Scripts that pass the first layer are run in a restricted environment. Dangerous functions and system access are blocked, and only safe operations like basic math or loops are allowed. This prevents malicious scripts from causing harm even if they bypass the static check.

Layer 3 OS Resource Limits:

Finally, the operating system enforces limits to prevent abuse, such as maximum memory, CPU time, number of processes, file size, and open files. These limits ensure that scripts cannot crash the system or use excessive resources.

4.4 Deployment Architecture

The gateway is deployed using Docker Compose with six main containers: PostgreSQL for the database, Keycloak for authentication, FastAPI for the backend API, a React-based frontend, Mailhog for handling one-time passwords and email testing during development, and Nginx as a reverse proxy. Secure communication is ensured with mutual TLS (mTLS) between Nginx and FastAPI, using TLS 1.3 exclusively. Nginx enforces HTTP Strict Transport Security (HSTS) and applies rate limiting of 10 requests per second to prevent abuse. All components are hosted on-premises to guarantee data sovereignty.

